



先進工学研究科 電子システム工学専攻

音響工学特論

第 1 1 回

植木 祥高

東京理科大学 先進工学部
電子システム工学科

授業計画

- #01 インTRODクシヨン
- #02 機械学習の歴史と動向
- #03 機械学習分野の問題・具体的手法
- #04 Pythonのプログラミング基礎
- #05 音響収録の基礎
- #06 ニューラルネットワークの基礎 ①
- #07 ニューラルネットワークの基礎 ②
- #08 ニューラルネットワークの実装 ①
- #09 ニューラルネットワークの実装 ②
- #10 ニューラルネットワークの実装 ③
- #11 ディープラーニングの概要と動向
- #12 音響データを用いた機械学習 ①
- #13 音響データを用いた機械学習 ②
- #14 プレゼンテーション
- #15 到達度評価

※ 評価 : 60%

※ 評価 : 40%

(機械学習・音響工学の知識と習得度を評価)

ディープラーニングの概要と動向

- ディープラーニングの代表的な手法について理解する

ニューラルネットワークの基礎

- 前回までのおさらい

ニューラルネットワークの訓練と性能評価

- 次なるステップ

ディープラーニングに対する理解の獲得

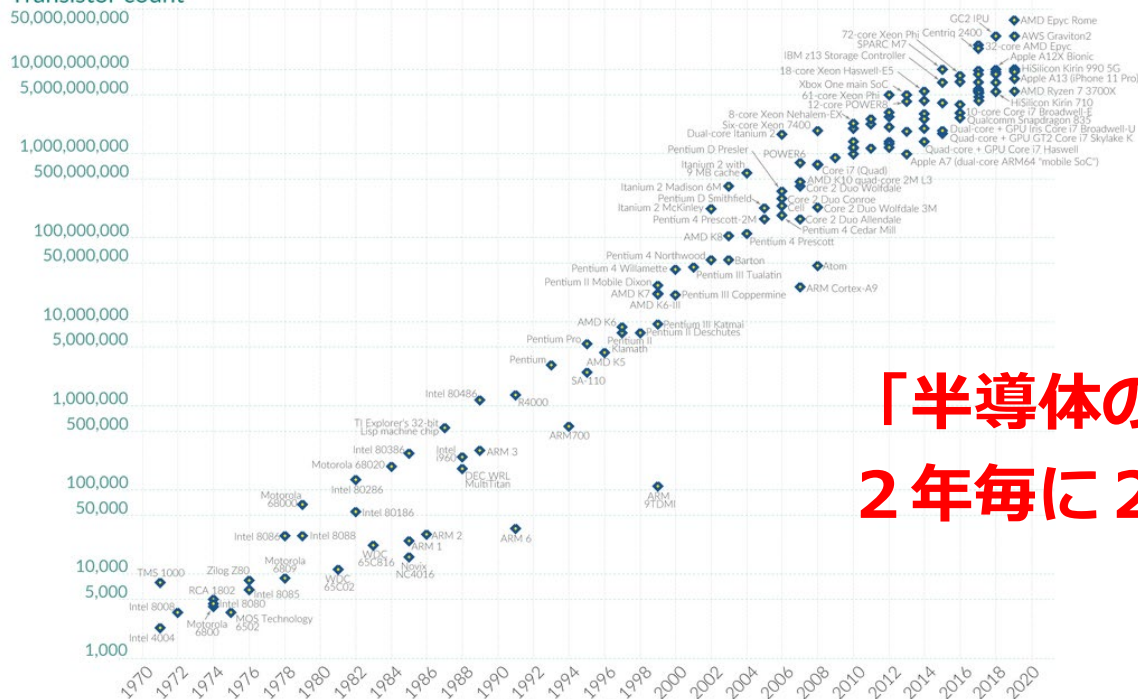
ハードウェアの進化

- 半導体の性能・集積が深層学習の進化を支えてきた
- 「ムーアの法則」 1965年に提唱された経験則

Moore's Law: The number of transistors on microchips doubles every two years
 Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

Transistor count



「半導体の性能と集積は
2年毎に2倍になる」

Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)
 OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

<https://ja.wikipedia.org/wiki/ムーアの法則>

演算処理装置

- CPU (Central Processing Unit)

コンピュータ全般の作業の処理を担当

- GPU (Graphical Processing Unit)

画像処理に関する処理を担当

General-Purpose GPU : 画像以外の目的使用に最適化

GPU

- ディープラーニング向けのGPUはNVIDIA社がリード
- DL実装用のライブラリのほぼ全てがNVIDIAのGPU上での計算をサポート



<https://www.nvidia.com/ja-jp/geforce/graphics-cards/40-series/rtx-4090/>

<https://www.nvidia.com/ja-jp/>

ディープラーニングのデータ量

- ネットワークが深くなれば、パラメータ数も増加
(AlexNetでは60,000,000個)
- データ量の目安の経験則 「バーニーおじさんのルール」

「モデルのパラメータ数の10倍のデータ数が必要」

- ※ 最近のDLに適用可能か不明。数学的な証明もない。
- ※ ただ、データ数が少なすぎる場合はDLの訓練は不十分なことは間違いない

活性化関数

- 層が深くなれば、誤差を逆伝播する際に勾配が消失
- 学習の鈍化が発生（勾配消失問題）
- 活性化関数の微分の最大値が原因

シグモイド関数のような勾配の最大値が0.25といった小さな値を持つ活性化関数は勾配消失問題を引き起こしやすいことが知られている



シグモイド関数



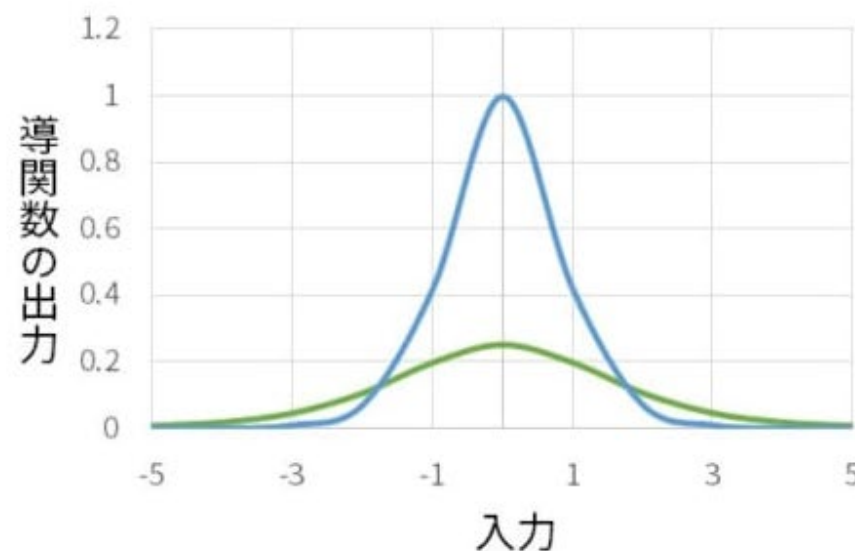
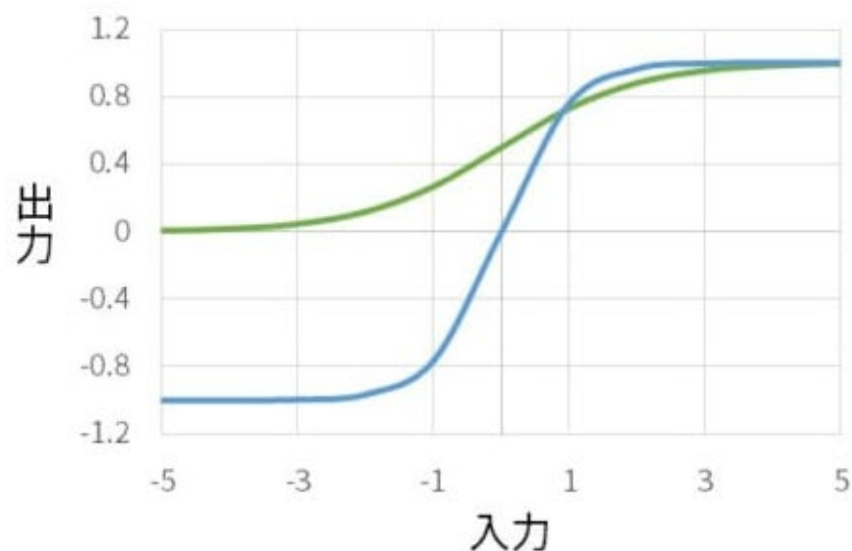
シグモイド関数の微分値（勾配）

活性化関数

- tanh関数

すべての入力を-1から+1の範囲で出力する活性化関数

■ tanh関数 ■ シグモイド関数



導関数の最大値がシグモイド関数より大きいため、勾配消失問題を比較的抑制します

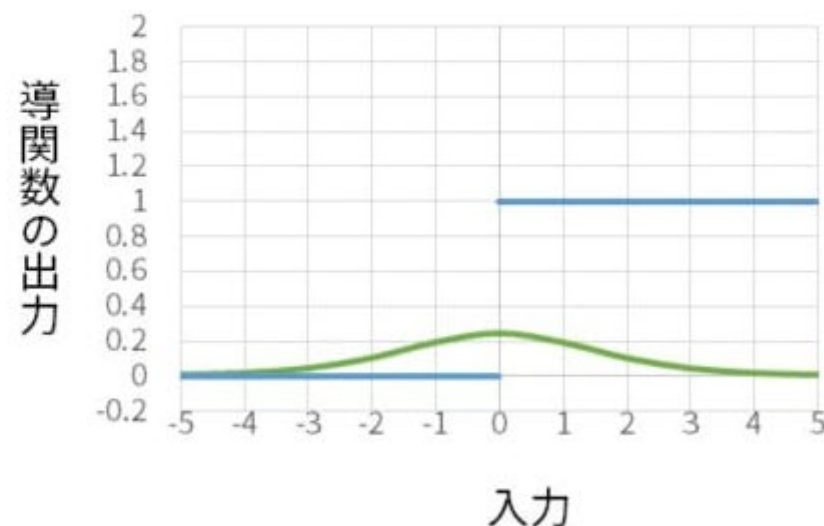
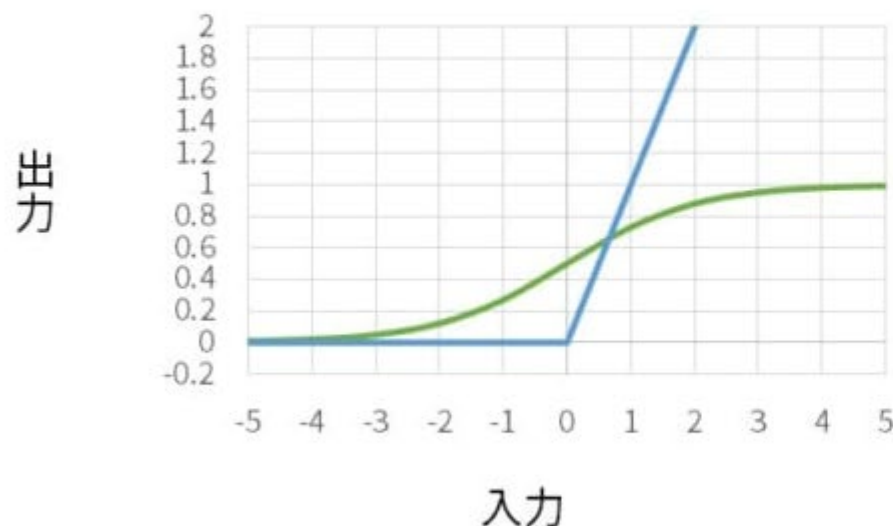
<https://zero2one.jp/ai-word/tanh-function/>

活性化関数

• ReLU (Rectified Linear Unit) 関数

負の入力を0、0以上の入力をそのまま出力する活性化関数

■ ReLU関数 ■ シグモイド関数



導関数の最大値がシグモイド関数より大きいため、勾配消失問題を比較的抑制します
また、計算が単純なため、学習において高速な計算が可能です

<https://zero2one.jp/ai-word/relu-function/>

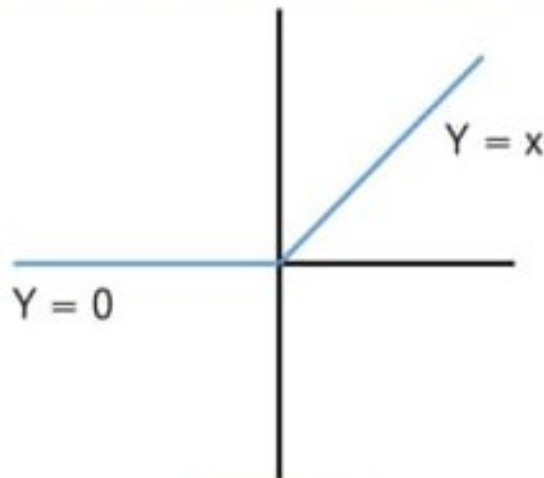
活性化関数

• Leaky ReLU 関数

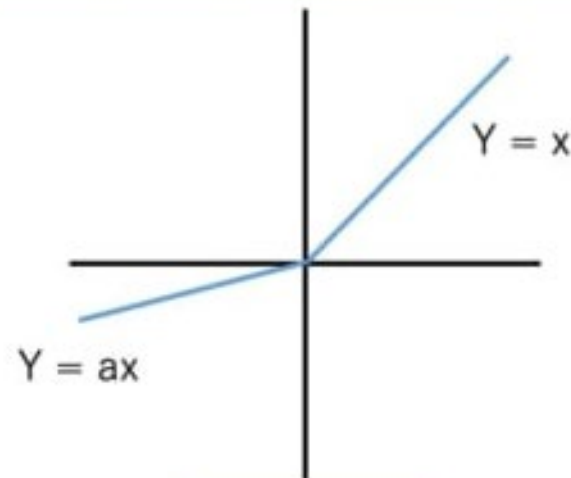
従来のReLUでは入力が0以下のとき勾配も0のため、学習を進めるうちに勾配が壊れやすい



少しだけ傾斜をつけることにより、この問題に対応したのがLeaky ReLU !



ReLU



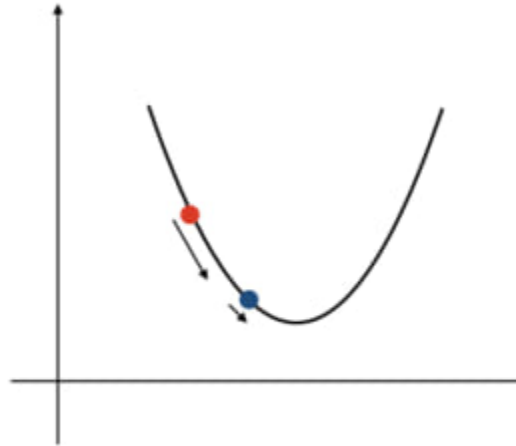
Leaky ReLU

© zero to one, Inc. All Rights Reserved.

<https://zero2one.jp/ai-word/leaky-relu-function/>

最急降下法

最適化の手法である勾配降下法のアルゴリズム。
関数の傾きから最小値を探索する方法。



最急降下法の手順

- ①初期値を決定 (図中●)
- ②その点における勾配を算出し、最急降下方向を計算
- ③移動幅を決定し、最急降下方向へ移動 (図中●)
- ④収束するまで②、③を繰り返す

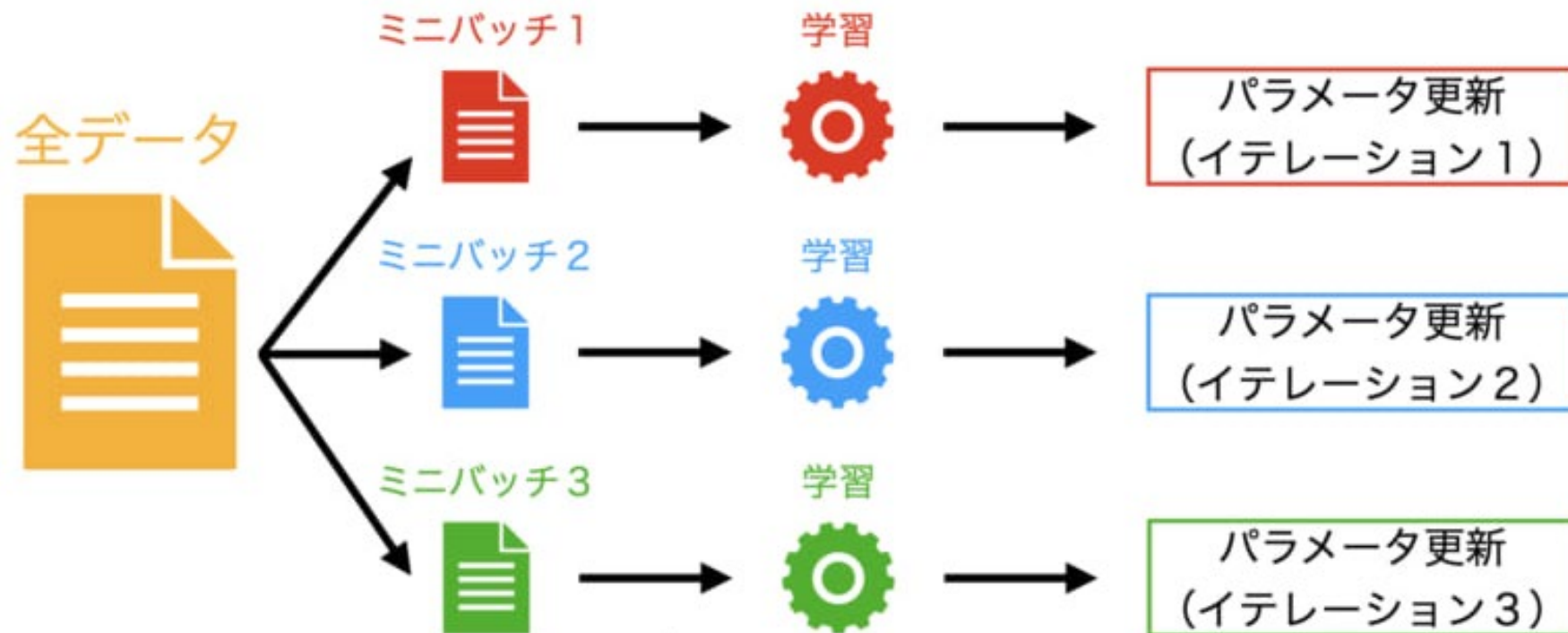
$$x^{(k+1)} = x^{(k)} - \alpha f'(x^{(k)})$$

学習率 $\alpha > 0$

k : イテレーション

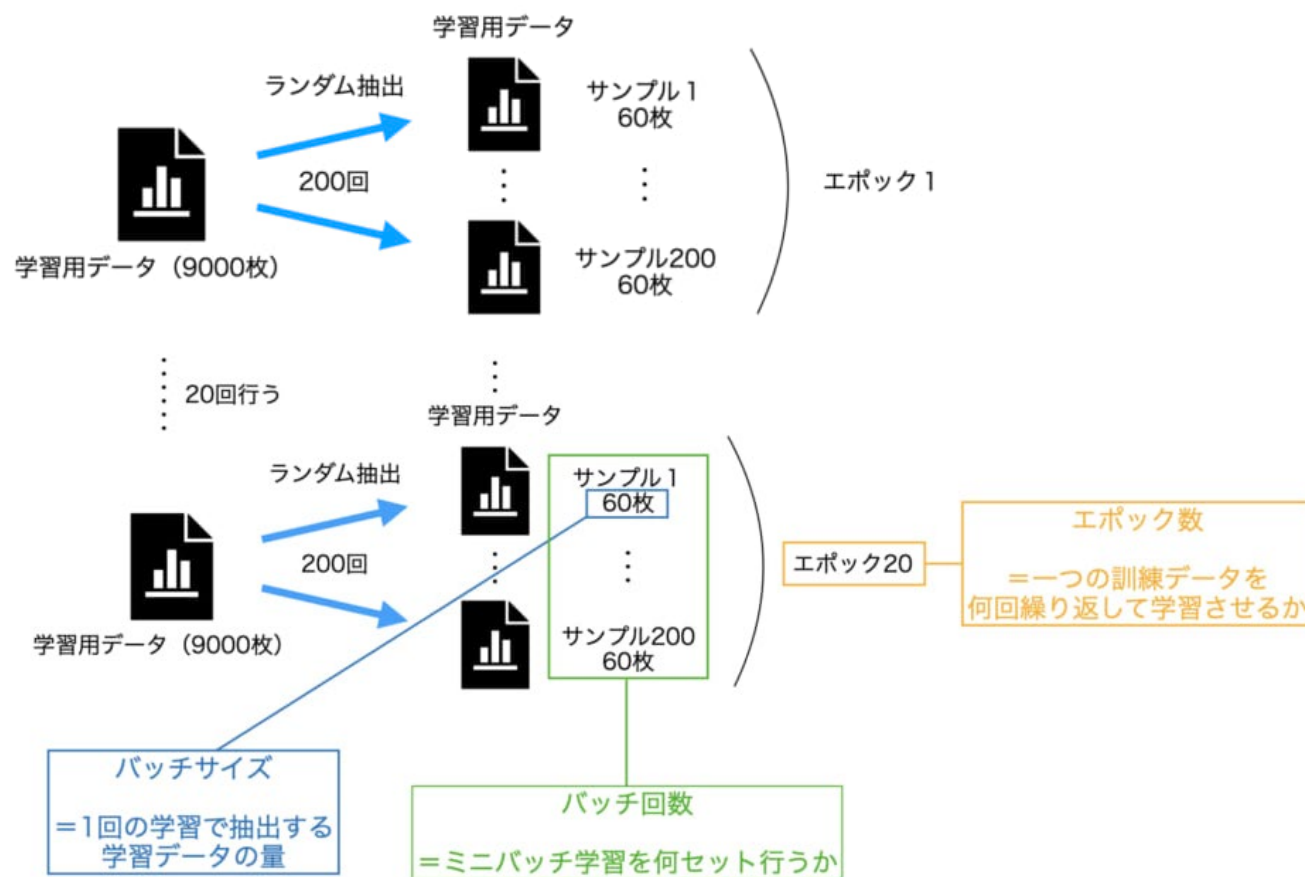
イテレーション

イテレーション＝機械学習において重みが更新された回数



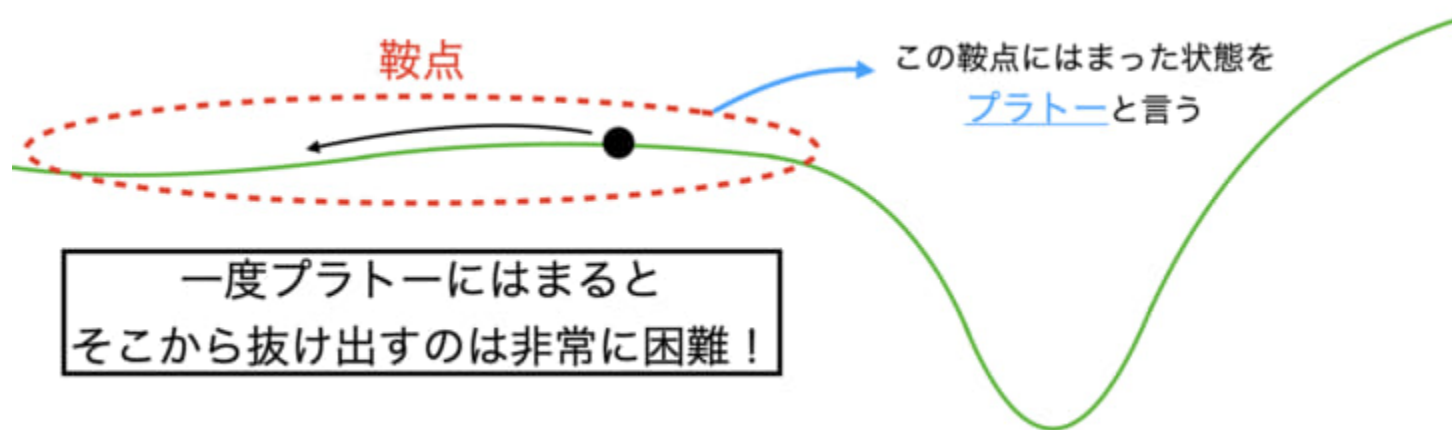
ミニバッチ学習

=パラメータの更新をサンプル1つ単位で行うのではなく、
少数のサンプルをひとまとめにし、その単位でパラメータを更新する手法



プラトー

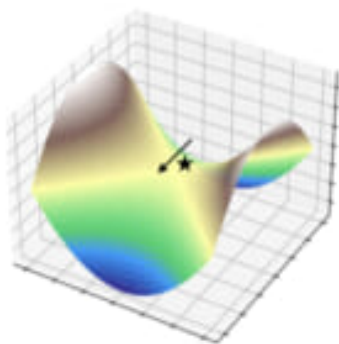
勾配降下法にて勾配の小さな平坦な地点（鞍点）に陥る状態



モーメントム

最適化の進行方向に学習を加速させる手法

※ 物理でいう慣性の考え方を適用

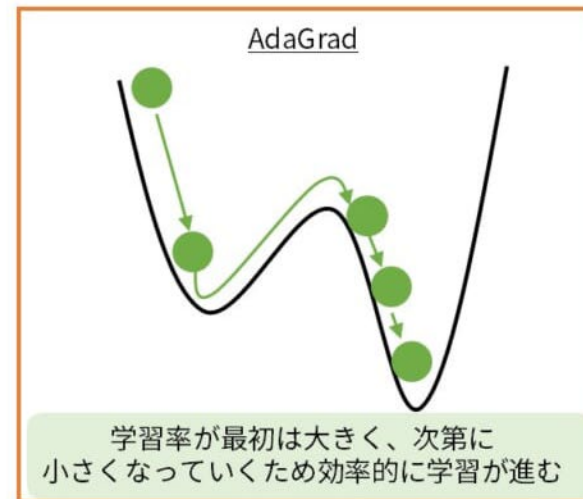
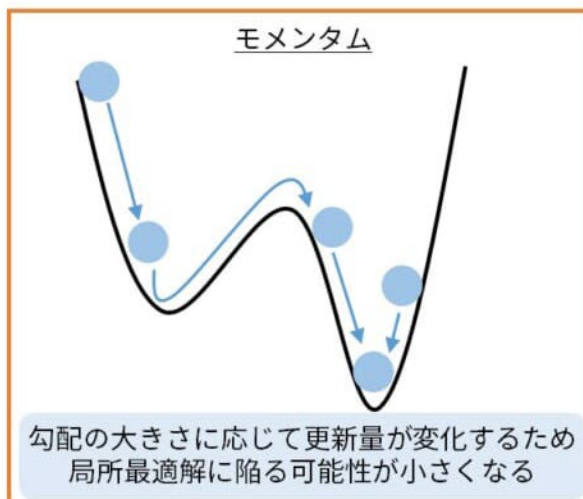
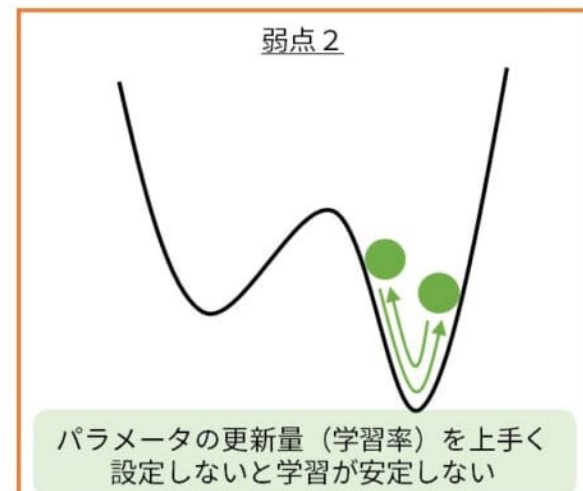
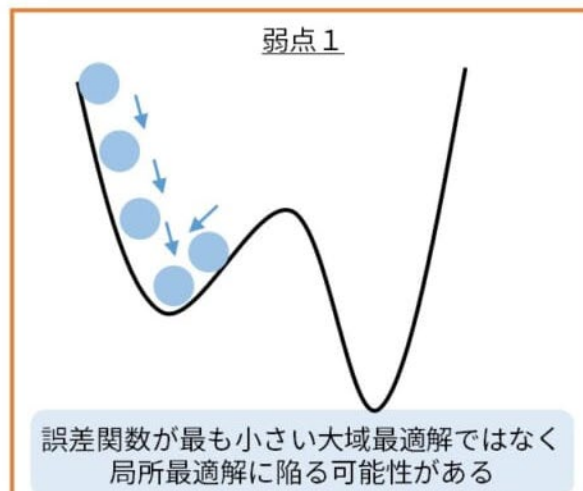


→の方向に加速させれば学習が再開する



鞍点での停留状態(プラトー)から脱出が可能

勾配降下法の改善



この二つの発想を取り入れた手法がAdamであり、汎用的に用いられています。

最適化手法

古いものから

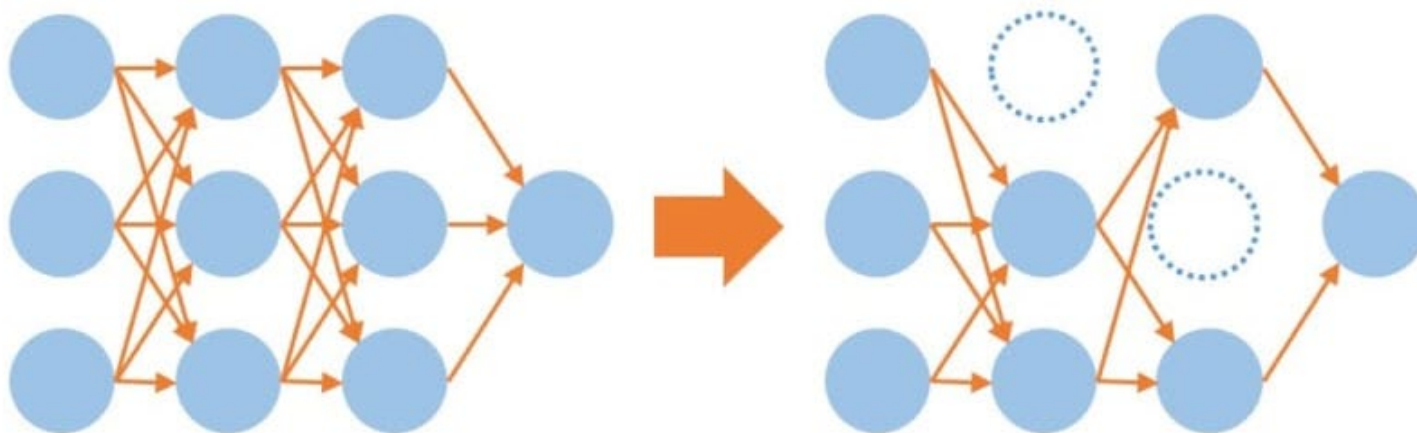
- Adagrad
- Adadelata
- RMSprop
- Adam

オーバーフィッティング

- 訓練データに対する過敏な反応
- 訓練データに対しては正確な予測
- 新規データに対しては不正確な予測を与える

ドロップアウト

ニューラルネットワークの学習時にノードをランダムで不活性にする手法



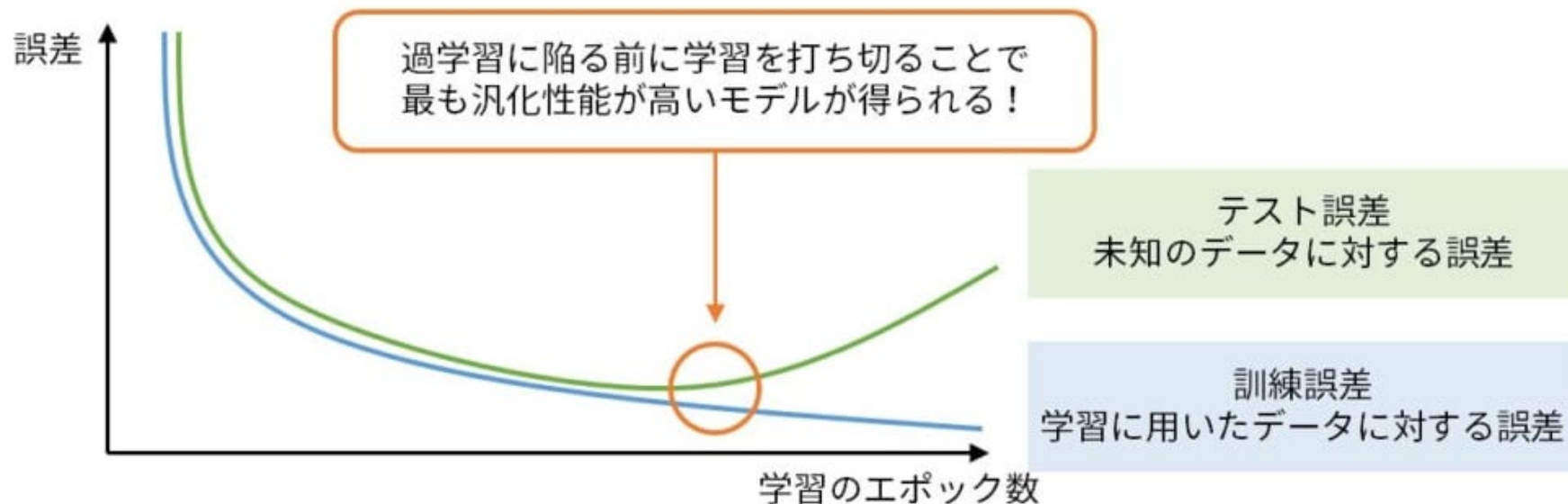
不活性になるノードは学習中一定という訳ではなく、各ミニバッチごとにランダムに選ばれます。

ディープラーニングはパラメータ数が多いためにモデルが過剰にデータセットに適合する過学習が
起こりやすいという課題を抱えていますが、ドロップアウトはこれをある程度抑制します。

- エポックごとに異なるネットワーク構造で学習
- 全体ではパラメータの値が調整される

早期終了（Early Stopping）

モデルが訓練データを過剰に学習し汎化性能が低下する「過学習」に陥る前に学習を終了させる手法



一般的に、学習のエポック数が多すぎると訓練誤差は下がってもテスト誤差は上昇します。

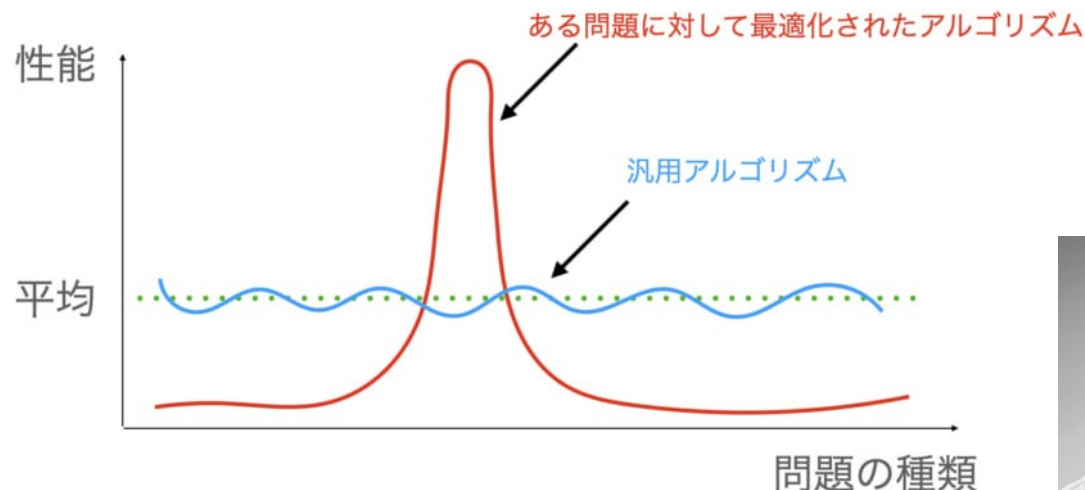
- 極めてシンプル
- どんなモデルにも適用できる (Beautiful FREE LUNCH)

<https://zero2one.jp/ai-word/early-stopping/>

ノーフリーランチの定理 (No-Free-Lunch Theorem)

- あらゆる問題に対して高い性能を示す汎用アルゴリズムは存在しないことを示した最適化領域の定理

特定の問題に対して最適化されたアルゴリズムも
汎用アルゴリズムも平均すると同程度の性能を示す



David Wolpert



Geoffrey Hinton

早期終了は
Beautiful
FREE
LUNCH

<https://zero2one.jp/ai-word/early-stopping/>

<https://www.santafe.edu/people/profile/david-wolpert>

<https://cifar.ca/bios/geoffrey-hinton/>

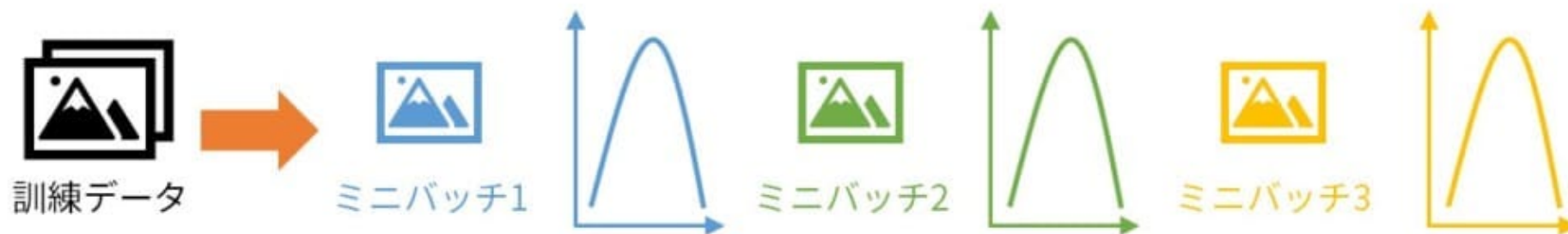
バッチ正規化

- 各層に伝わってきたデータを、活性化関数に通す前に再度正規化
- 学習の促進、オーバーフィッティングの防止にも

ミニバッチを平均0、標準偏差が1となるように正規化を行うことで学習を効率的に行う手法



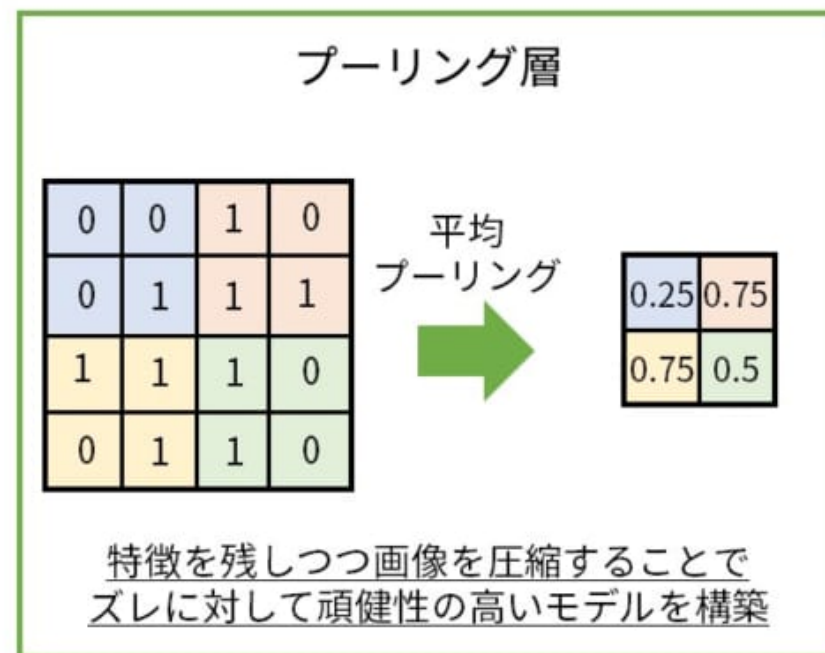
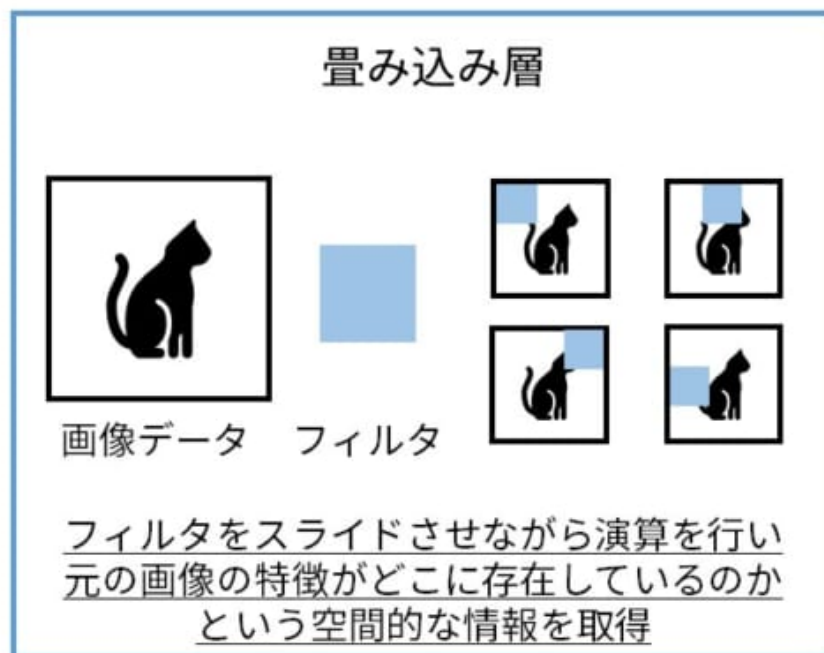
ミニバッチはランダムに選出されるため、毎回特徴量のスケールが異なる



バッチ正規化により学習がミニバッチのランダムな選出の影響を受けづらくなり学習が安定します。

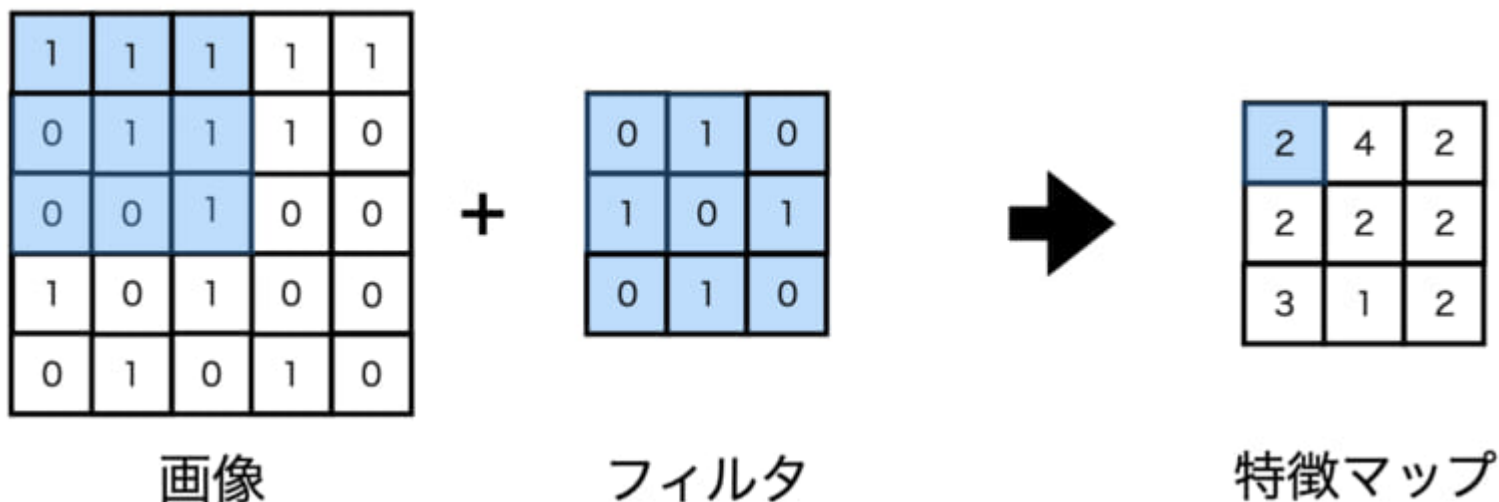
畳み込みニューラルネットワーク

特徴の位置を学習する「畳み込み層」と特徴の位置ズレを補完する「プーリング層」を繰り返したもの



- 画像をそのまま二次元で入力に用いることができるモデル

畳み込み



- 画像から特徴を抽出する操作
- フィルター（カーネル）を適用することにより抽出
- 一定間隔でスライドさせ適用し、フィルター値と入力値の積の和を出力する

プーリング

ストライド1

4	8	0	4	8
10	12	8	6	2
12	12	14	12	8
8	8	20	4	6
10	16	24	12	8

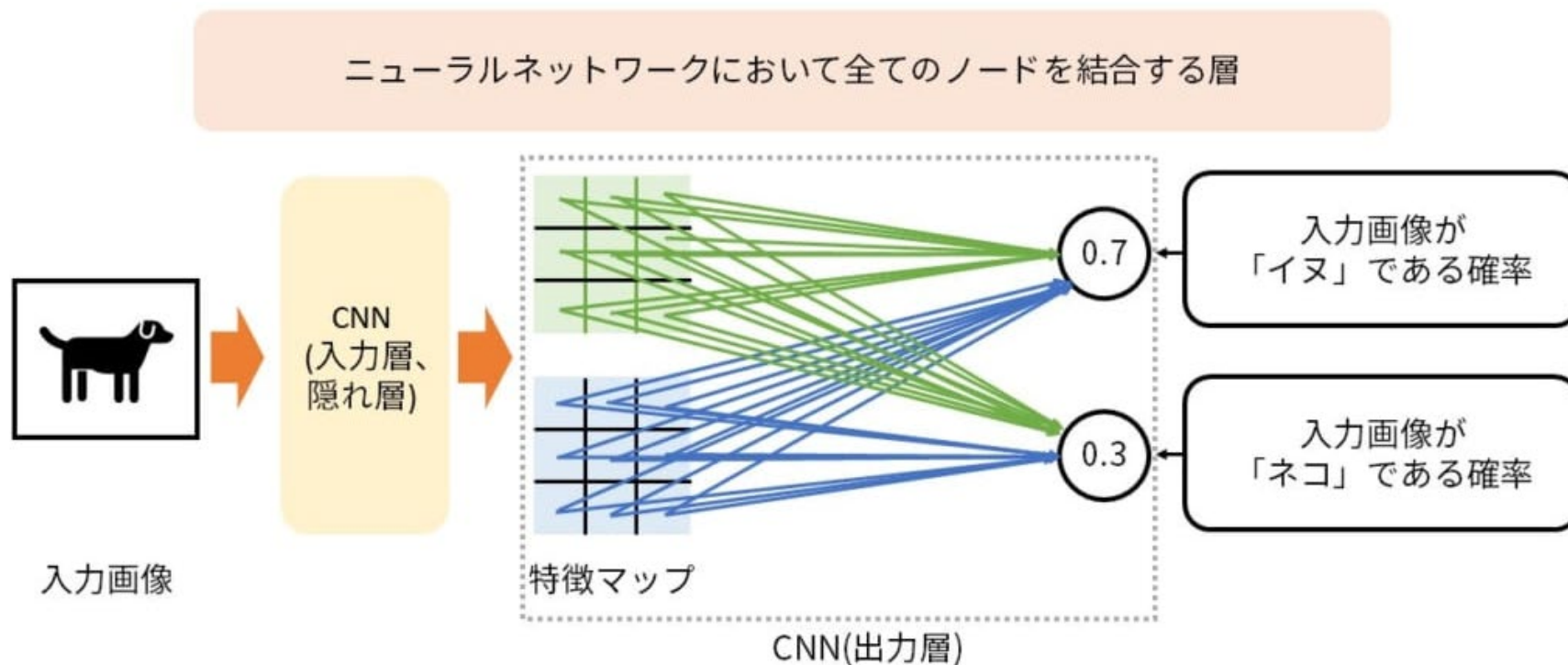
プーリング



12	12	8	8
12	14	14	12
12	20	20	12
16	24	24	12

- 画像から特徴を抽出する操作
- 画像サイズを決められたルールに従って縮小する手法
- maxプーリング（上図）、avgプーリング

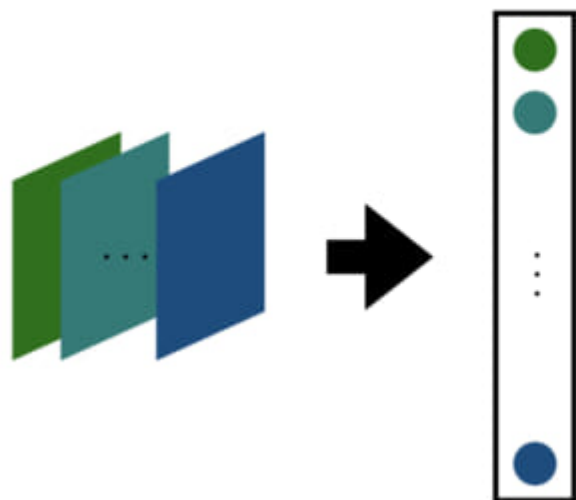
全結合層



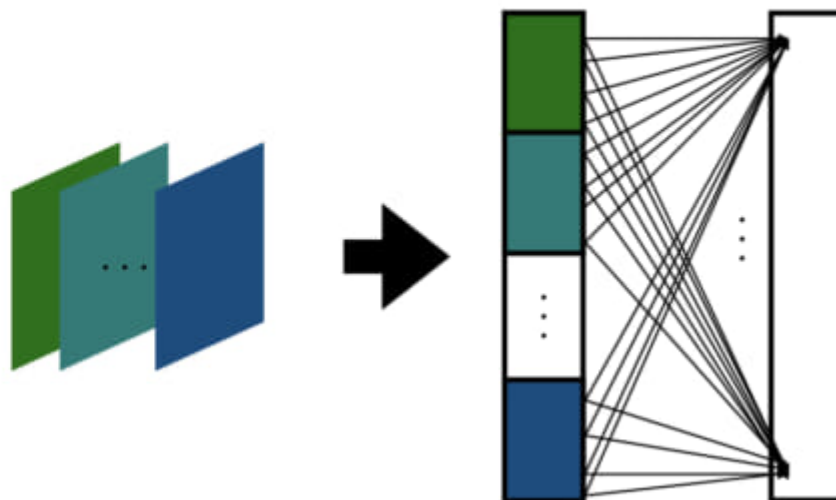
- CNNにおいて出力層には全結合層が用いられ、特徴マップを結合して出力
- 一次元の数値とすることでラベル分類においては「確率」に対応

グローバルアベレージプーリング

グローバルアベレージプーリング



全結合層



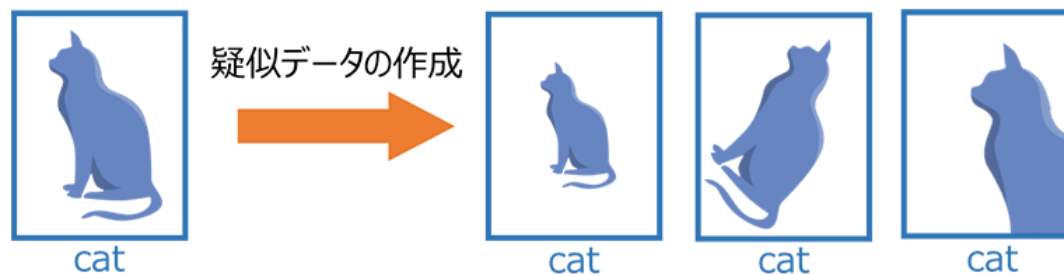
- CNNにおいて全結合層の代わりに、
「1つの特徴マップに1つのクラスを対応させる」ことで分類を行う
- 各チャンネルの全画素の平均値をとるため、VGG-16（512チャンネル）の場合、512個の重みパラメータでよく、メモリ使用量の低減が可能

移動不変性

- 畳み込みやプーリングは移動不変性を獲得するための処理
- 「位置のズレ」に強いモデル の構築に貢献
- 残された課題
 - ✓ 同じ物体でも、角度が異なれば画像として全く見え方が異なる
 - ✓ 拡大縮小によって見え方が異なる
 - ✓ 光の当たり方によって見え方が異なる

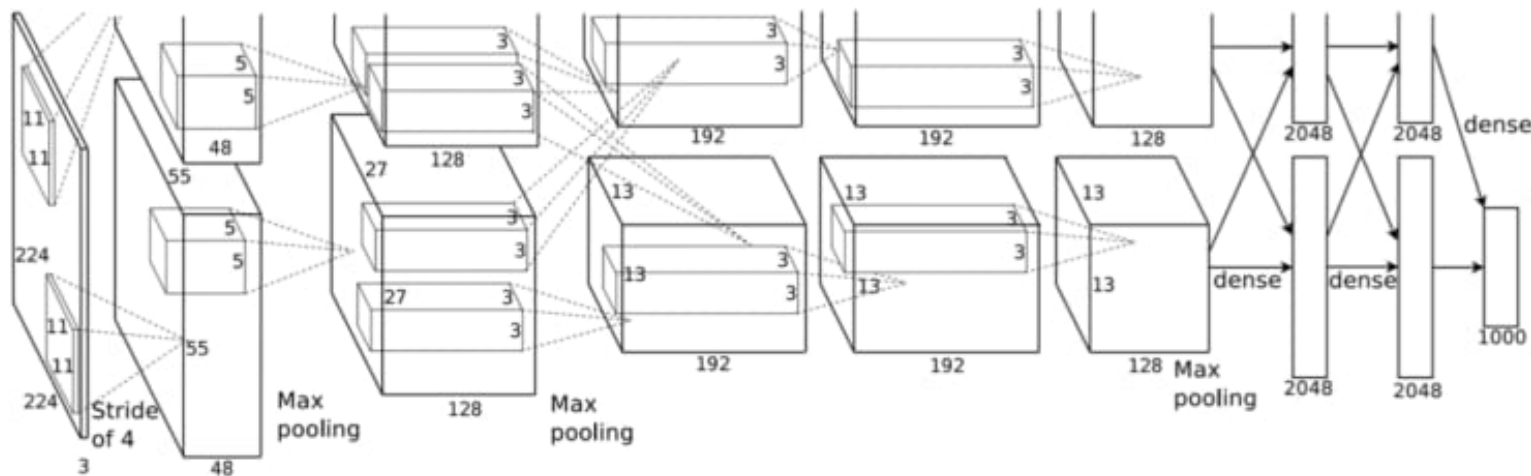
データ拡張 (Data Augmentation)

- データ収集の代わりに、所有データから疑似的に別のデータを生成するアプローチ
- ランダムにいくつかの処理を行い、新しいデータを作り出す
- 処理例
 - ✓ 上下左右にずらす
 - ✓ 上下左右に反転する
 - ✓ 拡大・縮小する
 - ✓ 回転する
 - ✓ 斜めにゆがめる
 - ✓ 一部を切り取る
 - ✓ コントラストを変更する



※ ただし、データ拡張には現実に即したものである必要あり

AlexNet (SuperVision)

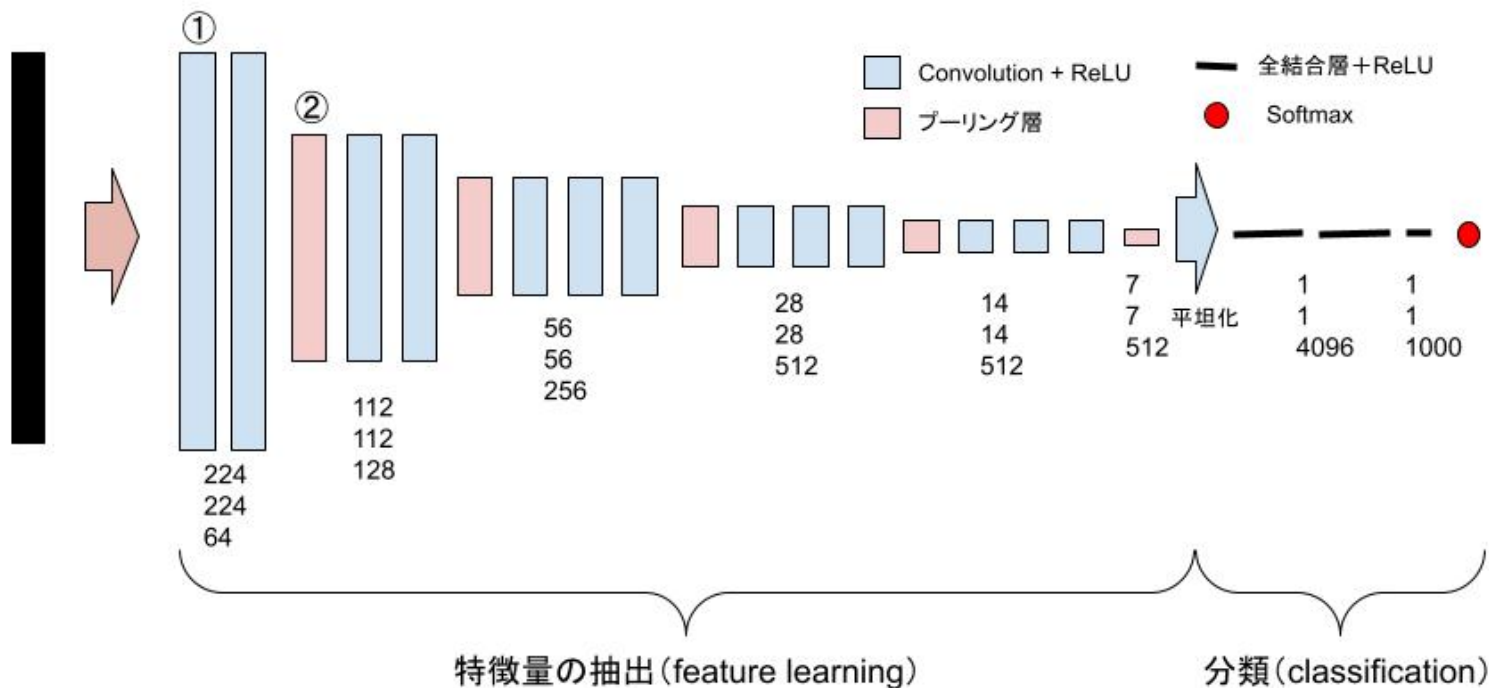


参考 : Krizhevskyら "ImageNet Classification with Deep Convolutional Neural Networks"

- 2012年にILSVRC（画像認識の精度を競う大会）で優勝
- 畳み込み層とプーリング層を交互に使用
- パラメータ数は6000万、活性化関数はReLU関数
- これ以降、畳み込み層とプーリング層を交互に使用した層の深いモデルが登場

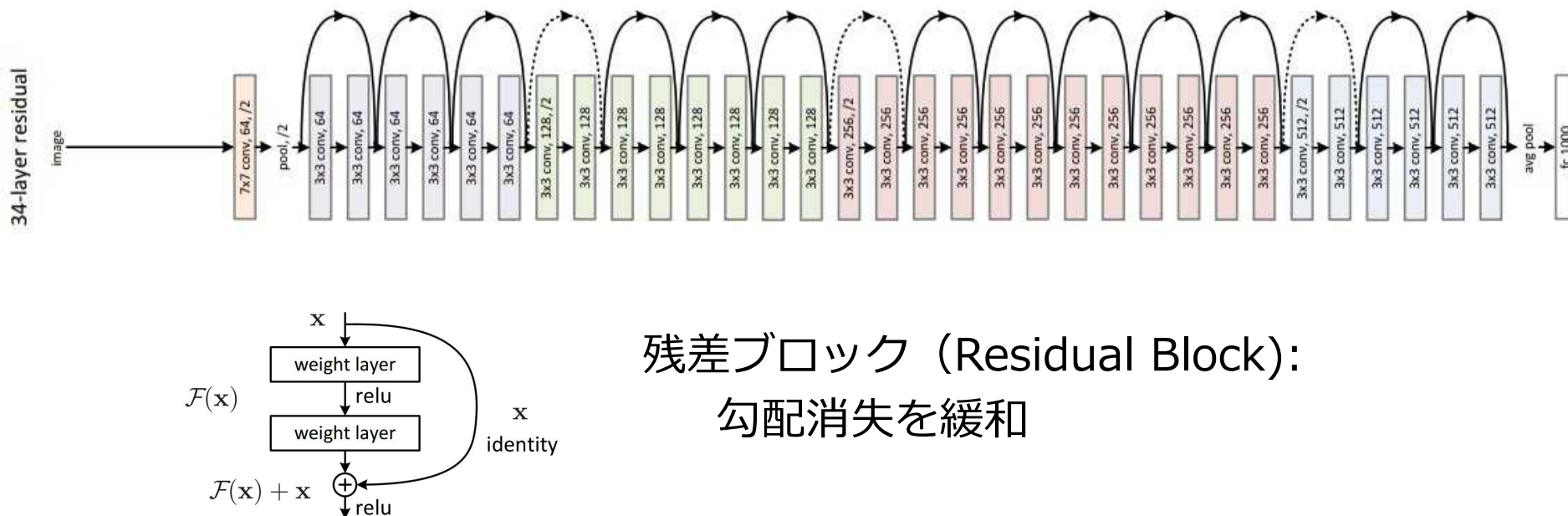
VGG

VGG16の構造



- 畳み込み層のカーネルサイズを3 x 3に統一
- プーリング後にチャンネル数を割り増しする

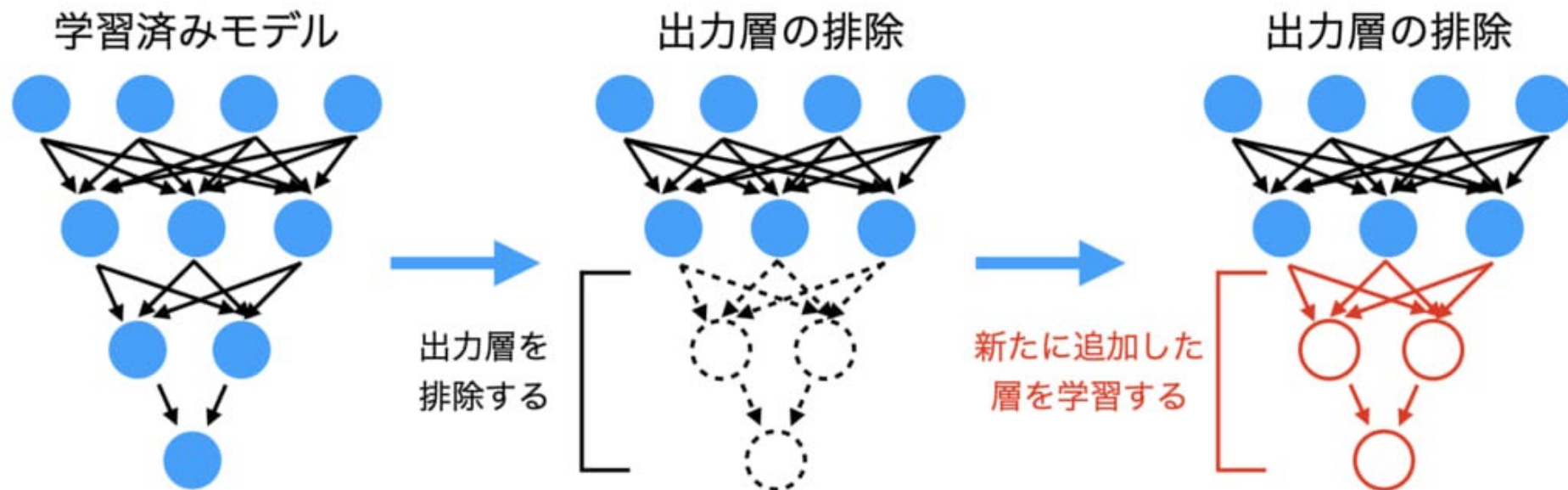
ResNet (Residual Network)



残差ブロック (Residual Block):
勾配消失を緩和

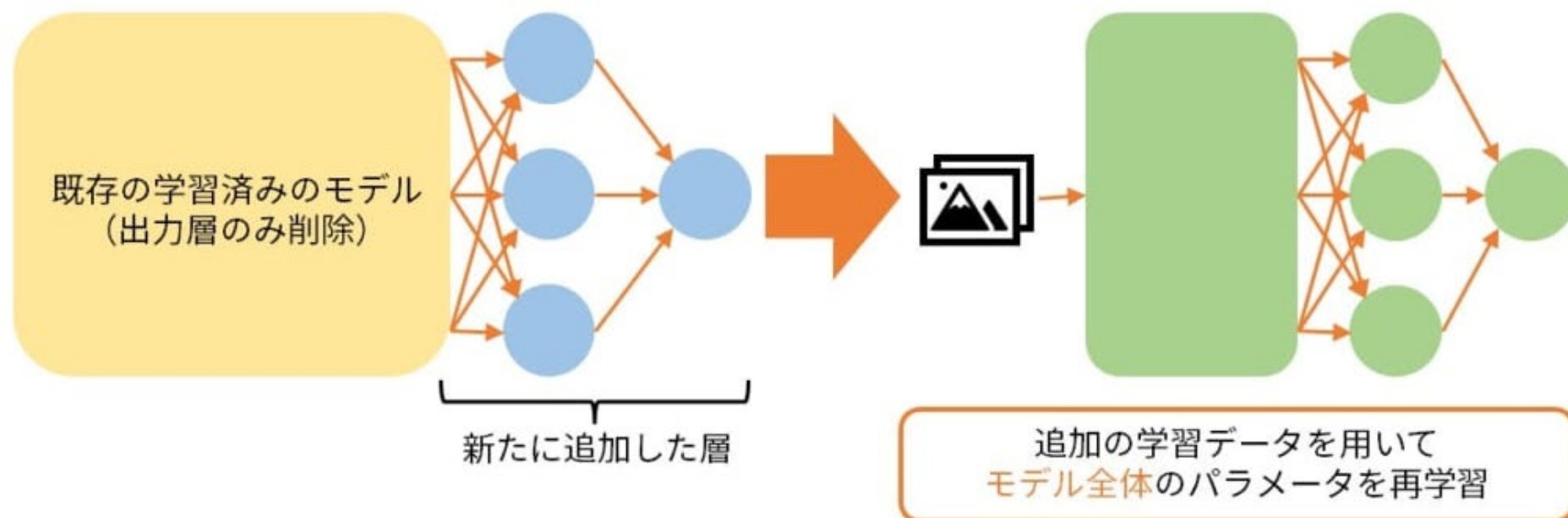
- 畳み込み層とskip connectionという層を飛び越えた結合を組合わせたネットワークアーキテクチャ
- 層を深くしても学習が進む

転移学習 (Transfer Learning)



- 「最適化されたネットワークの重み」を利用して新しいタスクの識別に活用
- 学習済みのネットワークが良い特徴量の抽出することを期待
- 出力層のみを変更し、変更箇所のみ学習を行う

ファインチューニング (Fine Tuning)



- 学習済みのモデルに新たな層を追加し、モデル全体を再学習
- 短時間での学習を可能に
- 追加のデータセットが充分用意できるときにファインチューニングを、そうでない場合は転移学習を採用するという考え方（※ケース依存）



Tokyo University of Science